UNIVERSIDAD AUTONOMA DE NAYARIT

Unidad Académica de Economía

Carrera

Licenciatura en Sistemas Computacionales

Grupo y Semestre

Semestre 6

Nombre estudiante

Ricardo Matos Vizcarra

Actividad

Reto IA 9

Maestro

Eligardo Cruz Sánchez

Materia

Programación Distribuida del lado Cliente

Muestra de los 5 logs reales

```
ryzenx@Ryzen:~/Documents/RepoGit/Programacion_Distribuida_del_lado_Cliente/Semana 1/Reto IA 9$ node cliente-HTTP-Logger.js | jq
--- Iniciando pruebas con Logs Estructurados ---

{
  "timestamp": "2026-02-27T21:02:00.277Z",
  "level": "INFO",
  "method": "GET",
  "url": "/productos",
  "status": 200,
  "duration_ms": 43,
  "size_bytes": 257,
  "message": "OK"
}
{
  "timestamp": "2026-02-27T21:02:00.278Z",
  "level": "DEBUG",
  "method": "GET",
  "url": "/productos",
  "status": 200,
  "duration_ms": 43,
  "size_bytes": 257
}
{
  "timestamp": "2026-02-27T21:02:00.285Z",
  "level": "ERROR",
  "method": "POST",
  "url": "/productos",
  "status": 400,
  "duration_ms": 7,
  "size_bytes": 105,
  "message": "El precio 12.555 no es válido. Máximo 2 decimales permitidos."
}
{
  "timestamp": "2026-02-27T21:02:00.285Z",
✗ Error de UI: Fallo al crear producto.
  "level": "ERROR",
  "method": "POST",
  "url": "/productos",
  "duration_ms": 7,
  "error": "El precio 12.555 no es válido. Máximo 2 decimales permitidos."
}
{
  "timestamp": "2026-02-27T21:02:00.287Z",
  "level": "ERROR",
  "method": "GET",
  "url": "/endpoint-que-falla",
  "duration_ms": 2,
  "error": "Formato no válido: {\"type\":\"https://sto"
}
ryzenx@Ryzen:~/Documents/RepoGit/Programacion_Distribuida_del_lado_Cliente/Semana 1/Reto IA 9$
```

Propuesta de 2 mejora para una Versión 2.0

Para escalar esta capa de observabilidad hacia un entorno de producción en la nube (ej. Datadog, AWS CloudWatch o ELK), implementaría las siguientes dos mejoras:

1. **Implementación de un x-request-id (Correlation ID):**

Generar un UUID único en el cliente antes de cada petición (crypto.randomUUID()) y enviarlo en los headers (x-request-id). Este ID se incluiría en el log JSON. Esto permitiría rastrear una solicitud específica a lo largo de todos los microservicios del backend para depurar fallas complejas sin perder el contexto.

2. **Contextualización de Entorno y Usuario (Tags):**

Agregar al objeto base del Logger etiquetas estáticas como env: "production" | "development" y dinámicas como userId (extraído del token JWT). Esto es crucial para poder filtrar rápidamente en los dashboards y saber si un error masivo está afectando a usuarios reales o si solo es tráfico de pruebas del equipo de desarrollo.